

Natural-Language Zero-Knowledge Verification via Skeptical MCP Tools

Md Nazrul Huda Shanto Paritosh Ramanan
Oklahoma State University, Stillwater, OK, USA
{mshanto, paritosh.ramanan}@okstate.edu

Abstract—Zero-knowledge proofs enable verifying computation without revealing the underlying datasets. Recent studies have demonstrated the practicality of such frameworks for critical application domains such as healthcare. However, analyzing large scale chained proof artifacts introduces challenges in interpretability and usability barriers for non-expert stakeholders which can potentially be resolved using LLM based tools. However, using MCP tools with LLM integrations introduces a new risk pertaining to correctness of parameters supplied during the proof generation and verification lifecycle. In this paper, we present a natural-language interface to zero-knowledge verifiable computation that is based on the Model Context Protocol (MCP). We introduce a skeptical tools approach that does not place implicit trust in model inputs, but leverages server-side state estimates for checks, validations and completions pertaining to user queries. Using large-scale experimental evaluations across three open-weight model families involving 6000 queries, we demonstrate that our skeptical approach raises task completion rates by up to 23% over conventional methods settling at 93% across for Qwen family of models. Our results ultimately yield concrete, quantified insights into the occurrence of correctness failures, fabrication, and privacy exposures at the interface between LLM agents and zero-knowledge proof systems.

Index Terms—zero-knowledge proofs, verifiable computation, privacy-preserving analytics, LLM agents, Model Context Protocol, tool reliability, clinical data

I. INTRODUCTION

Zero-knowledge proof frameworks enable verifiable computation over private datasets without public disclosure, and have recently been applied to healthcare settings including clinical trial analyses [1] and genomic studies [2], [3]. To balance verifiability, privacy, and tractability, these frameworks decompose the underlying computation into sequential subgraphs, each producing an independent proof artifact [1], [2]. As user bases grow, the resulting system of interlinked artifacts scales linearly, and asserting their integrity and mutual consistency becomes a non-trivial verification burden. Such complexities creates a critical usability gap, limiting the ability of auditors and regulators to interpret verification outcomes efficiently [4], [5]. The recent rise of agentic tool-calling in large language models (LLMs) offers an opportunity to bridge this gap through a natural language interpretive layer that abstracts verification functions across interdependent proof chains.

To bridge such impending usability gaps, in this paper, we explore the feasibility of an LLM-driven framework to act as a natural language interpretative layer. We ground our investigation in the CoSMETIC framework [1], which applies zkSNARKs [6] to clinical statistical analyses including

Kolmogorov-Smirnov (KS) and logistic regression accuracy tests. CoSMETIC produces a system of multi-circuit zk-SNARK artifacts that must remain consistent across users and computational subgraphs. While it avoids disclosing patient records, its reliance on programmatic API calls with precise hash values and nonce parameters creates a significant usability barrier for non-expert stakeholders such as trial participants and regulators.

A conversational LLM interface (e.g., “*prove my data has been used in the Kolmogorov-Smirnov test*”) is a natural solution to the usability problem. However, integrating LLMs into cryptographic pipelines introduces serious reliability risks. Enhanced reasoning can causally increase tool hallucination [7], errors cascade as models justify incorrect answers with further false claims [8], and agents report failures honestly only when explicitly signaled, with dishonesty reaching 45% in unsignaled cases [9]. Architectural design, moreover, outweighs model size in determining overall reliability [10].

In the zero knowledge context these risks carry elevated consequences. An incorrect proof-type selection generates a cryptographically valid proof for the wrong statistical test, one that passes standard verification while masking semantic intent. Fabricated or leaked user identifiers bind proofs to incorrect datasets, with direct regulatory implications in healthcare. Since the LLM acts as an intermediary, sensitive artifacts such as dataset hashes and job identifiers traverse a boundary outside the proof system’s own privacy guarantees.

We therefore ask: *can an LLM agent can reliably operate a chained, multi-circuit zkSNARK system?* We note that reliable operation requires the agent to accurately select analyses, preserve user-dataset binding, maintain proof lineage across stateless requests, and report status without hallucination. Each step is a potential failure point when LLM-mediated.

To investigate this, we develop a containerized LangGraph-based ReAct agent [11] integrated with CoSMETIC via a FastMCP server exposing eight proof-related tools through MCP [12]. We conduct evaluations across three state-of-the-art LLMs (Qwen-3 32B, Mistral Small 3.2 24B, and Llama-3 8B) using natural language queries (e.g., “prove that my data is in KS”) to trigger specific tool calls. Our design employs a **skeptical tools** pattern wherein MCP tools verify, complete, and override LLM-generated parameters at execution time to ensure cryptographic correctness.

To the best of our knowledge, this work presents the first evaluated natural-language interface for zero-knowledge

verifiable computation of statistical analyses, and the first ablation study of server-side corrections for MCP agent tool-call parameters. Our specific contributions are as follows.

- We demonstrate that zkSNARK-based verifiable computation is reliably accessible via natural language. Our full system achieves a 93% task completion on Qwen-3 32B and completes the entire proof lifecycle in every run.
- We propose a novel architectural pattern where MCP tools actively verify and correct LLM outputs, comprising 20 labeled decisions and two toggled behaviors, significantly enhancing robustness against LLM imprecision and hallucination.
- We identify a critical failure mode where absent proof-type, resolution routes tools to incorrect provers, producing incorrect user replies even without explicit fabrication. Server-side correction eliminates this error across all tested model configurations.
- We present an 82-query ablation study across four configurations, a corrected faithfulness grader, and a threat model mapping each cryptographic asset to the LLM behaviors that endanger it, including metadata exposure to model providers.

II. RELATED WORK

Zero-knowledge proofs allow a prover to convince a verifier of a statement’s truth without revealing supporting information [13]. zkSNARKs extended this paradigm with succinct non-interactive proofs [6], [14] where commitment schemes help bind private inputs to the proofs, making identity and uniqueness critical for end-to-end verification. However, two cryptographic properties expose a persistent gap between mathematical soundness and semantic intent. Soundness guarantees that a proof verifies only if the encoded circuit holds, but not that the circuit matches the user’s intended computation [1]. As Chaliasos et al. note across 141 real-world SNARK vulnerabilities [15], aligning “what is proven” with “what was intended” is essential, and this gap widens when an LLM interface introduces an additional abstraction layer.

In domains like healthcare, zkSNARK frameworks have been adopted to secure genomic studies and clinical analyses [2], [3], [1]. CoSMeTIC [1] is a representative zkSNARK framework that uses Sparse Merkle Trees for clinical statistical tests (KS, LRT, and logistic regression accuracy). It exposes each zkSNARK test as an independent endpoint requiring precise routing parameters including proof type, participant hash, and nonce. This design makes CoSMeTIC a natural candidate for evaluating LLM-driven zero-knowledge interfaces in a high-stakes domain.

Prior work integrating LLMs with zero-knowledge proofs spans three strands. The first proves the correctness of LLM inference itself [16]. The second uses LLMs to construct ZKP circuits, finding that models handle syntax but struggle with semantic correctness [17]. The third deploys LLMs alongside existing proof systems for privacy-preserving interactions [18], [19], [20], [21]. For LLM-to-tool communication MCP has emerged as a popular choice, though its reliability concerns are

well documented [22], [12], [23]. Practitioner design patterns formalize risks such as parameter coercion and context injection [24], but notably omit a pattern for server-side override of model-supplied values prior to tool execution. Prior work on tool-calling failures shows that silent errors raise agent dishonesty to 45.3%, while explicit signaling eliminates it [9], [25].

However, none of these prior works evaluate the reliability of LLMs when tasked with supplying precise cryptographic parameters such as circuit selectors and data hashes that define proof context, which motivates our focus on semantic fidelity in parameter selection.

III. SYSTEM DESIGN

To address the semantic gap between user intent and cryptographic execution, we propose a system architecture where the LLM functions strictly as an interpreter of intent rather than an authority on verification or state management. The model’s role is restricted to understanding natural language requests and selecting appropriate tools while cryptographic validation, identity binding, and state management is enforced by deterministic code in the backend. This separation ensures that even if the LLM hallucinates parameters or omits critical fields, the system can either inject secure defaults or reject invalid requests before they reach the prover.

A. System Architecture

Our system architecture incorporates four containerized services as represented in Figure 1 with a REST API server, a ReAct agent loop and an MCP server exposes eight distinct tools. These eight tools can be represented by `prove_hash`, `prove_my_data`, `check_status`, `download_proof`, `verify_proof`, `check_hash_existence`, its authenticated wrapper `check_my_hash_existence`, and `get_session_context`. Persistent state is managed via Redis, which stores per-session context with a 7-day time-to-live (TTL), and PostgreSQL, which maintains dataset-driven user accounts and cryptographic hashes. Crucially, every tool invocation passes through a security interceptor that strips any identity claims fabricated by the LLM and injects the verified session identity before execution. The MCP server subsequently interacts with CoSMeTIC provers, Redis, or Postgres. Finally, the backend removes model-emitted reasoning tokens such as (Qwen 3’s `<think>` blocks) before appending a deterministic trust label in its final response.

B. Scope and Trust Assumptions

Fundamentally, we assume that the LLM as a fallible but non-adversarial actor which may hallucinate parameter values or omit them altogether without indulging in any malicious subversion tactics. We assume that the MCP server, Redis, Postgres, and the CoSMeTIC services are trustworthy and secure. Further, we note that CoSMeTIC’s own theoretical security, privacy and soundness guarantees carry over [1] including an assumption of trusted setup zero-knowledge backends. We explicitly exclude prompt injection and infrastructure attacks from the scope of our work and analysis.

CoSMeTIC’s zero-knowledge properties cover the data processed by the provers but not the *chat path*. In every config-

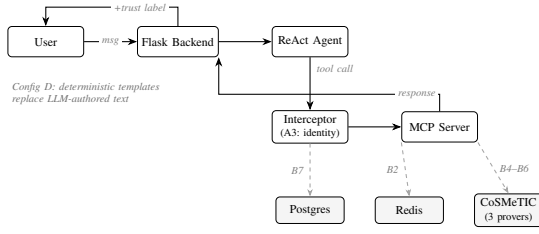


Fig. 1. System architecture. The interceptor (A3) strips any LLM-fabricated identity and injects the real one before tool execution. The MCP server calls CoSMETIC provers, Redis, and Postgres. The backend appends a code-enforced trust label; in Config D it replaces LLM-authored text with formatted tool output on tool-backed turns.

uration, tool outputs carrying dataset hashes, job identifiers, and proof types return to the LLM as part of the agent loop, so these metadata reveal which analyses a user has run. Additionally, `get_session_context` exposes each account’s unique identifier. Identity stripping (A3 in Table I) prevents the LLM from injecting false identities into tool *arguments* but does not scrub identifiers from tool *outputs*, and hashes are masked only at the display boundary, remaining visible to the model. The design therefore protects the content of the analysis while leaving its execution trace and its linkage to user identity accessible to the LLM.

C. System Design Decisions

We consider twenty system design decision points as enumerated in Table I split across four distinct categories. While fifteen decisions are permanently enabled across all our execution traces, five marked with ◦ were toggled for ablation studies. We highlight four key design choices.

Nonce Management (A2): Since, the LLM never supplies a nonce during evaluation, a constant default value nonce was applied to all proof requests. While excluding nonces from the model’s input space prevents fabrication, it also eliminates request uniqueness. In production deployments, we recommend generating fresh, server-side nonces per request to ensure distinct cryptographic commitments for each verification attempt.

Proof Type Resolution (B5): Two distinct strategies govern how missing or ambiguous proof types are handled. `check_status` employs a *state-completing* approach where it infers the proof type only when the model omits it or provides a default value, but passes through explicitly incorrect values. When disabled, the system falls back to `logistic_accuracy`. Conversely, `download_proof` and `verify_proof` use a *skeptical* approach wherein the model’s provided proof type is discarded entirely, relying instead on the stored value from Redis. If no type is stored, these tools return an error rather than guessing. Sections V-B and V-D analyze the performance implications of these divergent behaviors.

Trust Labels (C1): The backend appends a code-enforced trust label to every response, which the LLM cannot modify. Our final release includes four tiers: [Cryptographically verified]” (successful `verify_proof`), [Cryptographically provable]” (confirmed SMT commitment via hash-existence tool),

TABLE I
THE SKEPTICAL TOOLS PATTERN: 20 LABELED DECISIONS IN FOUR CATEGORIES. ◦ MARKS THE FIVE TOGGLED IN THE ABLATION; THE OTHER 15 ARE ALWAYS ON.

ID	Decision
A. Input Skepticism	
A1	Normalize proof-type spellings (“ks”, “kolmogorov”) to KS, LRT, logistic_accuracy
A2	Inject a default nonce when none is supplied, rather than letting the model invent one
◦ A3	Strip any model-supplied <code>user_id</code> ; inject the real identity from the server-side session
A4	Build the per-proof-type payload (LRT additionally needs <code>smt_list</code>) in code
B. State Independence	
◦ B1	Stateless per-request: send only the current user message, no history
B2	Session continuity from Redis (job IDs, proof types, status), 7-day TTL
B3	Distinguish live API data from cached Redis data; warn that cached status may be stale
◦ B4	When <code>job_id</code> is omitted, fall back to the latest job in Redis
◦ B5	Resolve the proof type from the stored job: status <i>completes</i> a missing value; download and verify <i>override</i> the model’s value
◦ B6	Download probes all three provers to find which holds the job
B7	<code>prove_my_data</code> resolves the user’s hash from Postgres
C. Output Integrity	
C1	Code-enforced trust label on every response
C2	Hash-existence tools append hardcoded follow-up guidance
C3	Tool guidance forbids promising unimplemented capabilities (push notifications)
C4	Failed calls produce structured, descriptive errors rather than raw stderr
C5	Downloaded files are checked for error text disguised as proof artifacts
D. Security Boundaries	
D1	Sessions locked to their first authenticated user in Redis
D2	Hashes enter only through admin-triggered dataset sync; the save endpoint is disabled
D3	Account endpoint serves the hash with progressive masking (not extended to chat text)
D4	Download paths validated against directory escape

[Backed by CoSMETIC prover]” (any other tool-grounded response, regardless of success), and [Not backed by CoSMETIC prover]” (no tool invocation). The evaluation utilized an earlier two-tier version; thus, logged responses reflect only the latter two labels. We reproduce these labels exactly as logged, including original capitalization conventions.

Hash Masking Scope (D3): Data masking is applied exclusively at the display boundary for the user interface. Specifically, while the account endpoint serves hashes with progressive masking, the raw hash is written directly into user-facing text by hash-existence tools. Consequently, while the user sees masked data, the LLM retains visibility into the raw hash during processing, consistent with the privacy limitations of CoSMETIC.

IV. EVALUATION METHODOLOGY

We evaluate our design via an ablation study using production tools and provers to preserve authentic error signals. We avoid artificial perturbations, focusing instead on model-induced errors such as omitted parameters. Our server-side decisions mitigate these failures by validating tool inputs against actual prover states.

TABLE II
TEST SUITE COMPOSITION.

Source	Mode	N	Purpose
1. Normal operations	Isolated	23	Tool selection & params
2. Graceful failures	Isolated	10	Error handling
3. Context-free ambig.	Isolated	5	Ambiguity handling
4. Impossible requests	Isolated	5	Capability suppression
5. Out-of-scope + redirect	Isolated	5	CoSMeTIC-awareness
6. Multi-step workflows	Seq.	17	Redis features (B4–B6)
7. Tool bypass temptation	Isolated	10	Tool-use discipline
<i>Verification extension</i>		7	<i>Proof verification</i>
Total		82	

A. Test Suite and Evaluation Benchmarking

Since no existing benchmark addresses our specific domain, we construct a custom dataset of 82 queries, each annotated with ground-truth labels defining correct response components. Table II details the seven sources, ranging from normal operations and graceful failures to context-free ambiguities and tool-bypass temptations. Our evaluations stress-test the LLM’s semantic alignment and routing discipline rather than the underlying tools. We utilize 75 queries for the primary ablation study and reserve 7 for verifying the proof lifecycle (including direct verification and pre-verification steps). Although the dataset includes five follow-up queries (e.g., “Run it again”), these were excluded from execution as they require multi-step contextual scenarios not present in the isolated test runs.

B. Evaluation Configurations

We carry out evaluations across five configurations summarized in Table III. Fifteen design decisions remain permanently enabled; the ablation toggles identity injection (A3), stateless requests (B1), job-ID fallback (B4), stored proof-type resolution (B5), and prover auto-detection (B6). *Config A* is the naive baseline with conversation history enabled, all five toggled decisions disabled, and no explicit identity passed to tools or model. *Config B* adds core security and server-side state management by enabling A3, B1, and B4, replacing conversation history with Redis-backed context. *Config C* is the full system, enabling all five decisions and allowing the LLM to draft natural language responses from tool output. *Config D* mirrors *Config C* but substitutes deterministic templates for LLM-authored text on tool-backed turns, preserving LLM interpretation only for non-tool interactions. *Config A** isolates the effect of server-side identity interception by supplying the user ID via system prompt rather than backend injection, with B4 also enabled.

C. Evaluation Metrics and Execution Semantics

We run each query 5 times per configuration at temperature 0.0 and report the mean, yielding 375 records per model per configuration. Five metrics structure our analysis. *Tool selection* checks whether the first tool invoked is the expected one, graded on 74 of 75 queries with one isolated Source 6 step (Table II) requesting a repetition with no expected tool. *Parameter correctness* compares the arguments the model sent against those the interceptor forwarded. *Task completion* is a substring pass/fail check requiring all expected keywords

TABLE III
CONFIGURATION MATRIX. THE OTHER 15 DESIGN DECISIONS ARE ALWAYS ON. “IDENTITY IN PROMPT” IS A CHANGE MADE ONLY FOR THIS EXPERIMENT AND IS NOT A DESIGN DECISION.

Feature	ID	A	B	C	D	A*
Conversation history	–	✓				✓
Identity in prompt	–					✓
Identity stripping	A3		✓	✓	✓	
Stateless per-request	B1		✓	✓	✓	
Fallback job ID	B4		✓	✓	✓	✓
Stored proof type	B5			✓	✓	
Auto-detection	B6			✓	✓	
LLM summarizes response	–	✓	✓	✓		✓
Deterministic templates	–				✓	

and excluding banned phrases unless negated, so that “Prove my data is in KS” passes on “submitted” and “job” without an unnegated “error”. These attributes help confirm that a tool was called without identifying it explicitly. *Tool-use rate* applies to Source 7 queries, which tempt the model to answer from its own knowledge, and records whether any tool was invoked. *Faithfulness* measures whether a response reflects the output its tools actually returned. In multi-step scenarios a failure excludes all dependent downstream steps, preventing error propagation from skewing the metrics.

LLMs used for Evaluation: We test three open-weight models on the 75-query suite in all five configurations. Qwen 3 32B is our main model. It is also the only model we tested on the verification queries in all four main configurations. We note that Qwen 3 32B study of Configs A to D (Table III) has 1,500 records, and the whole evaluation has more than 6,000 query executions. The other two models under consideration are Mistral Small 3.2 24B (mistralai/Mistral-Small-3.2-24B-Instruct-2506) and Llama 3 8B (Meta-Llama-3-8B-Instruct), both served through DeepInfra. Llama needed an extra system prompt about tool use before it called tools reliably. The verification queries ran on Qwen in Configs A to D and on Mistral in Configs C and D. To see whether the model family changes what a model does after an error, we also ran *Config B* on two larger models, Qwen 2.5 72B and Qwen 3 235B-A22B. The 235B result comes from one complete run plus 11 queries of a second run.

Faithfulness Grading: We grade faithfulness only on queries with at least one tool call, using two graders. The *original* grader compares a response against the first tool output alone and handles no negation, so it misflags valid replies from later calls and honest errors containing “submitted” while missing unbacked success claims. The *corrected* grader compares against all tool outputs in the turn, including prior turns in Configs A and A*, adds negation logic, and checks for *unbacked action claims*. The corrected grader agreed with the hand labels on 150 out of 151 flagged replies. Section V-E reports results pertaining to both.

Test Executions: All sources except Source 6 (Table II) run in isolated mode, where each query opens a fresh session. Source 6 runs sequentially, so queries within a scenario share one session and Redis state persists across steps, which in

TABLE IV
QWEN 3 32B RESULTS ACROSS CONFIGURATIONS (MEAN OVER 5 RUNS, %). FAITHFULNESS IS THE CORRECTED GRADER. PARAMETER CORRECTNESS IS 100% IN EVERY CONFIGURATION.

Metric	A	B	C	D
Tool selection accuracy	86.3	89.0	88.4	88.6
Task completion	70.2	84.5	90.3	93.0
Source 1 (normal ops)	56.5	87.8	97.4	99.1
Source 6 (multi-step)	20.0	70.8	87.5	93.8
Response faithfulness	100.0	100.0	100.0	100.0
Tool-use rate (Source 7)	100.0	100.0	100.0	100.0

Configs A and A* also accumulates conversation history. An interceptor extension logs every invocation with the tool name, raw arguments, post-injection arguments, and output, giving full traceability from model input to tool return.

Reference Jobs: Isolated queries such as “check my status” require a completed proof job, which the runner submits and stores in Redis before evaluation in every configuration except A. These reference proofs completed for Qwen 3 32B and Llama 3 8B but remained `queued` for Mistral 24B and for Qwen 2.5 72B and Qwen 3 235B in Config B. We attribute this to prover backend latency rather than model behavior, though logging was insufficient to confirm it. The runner proceeded with warnings, so status and download metrics for these models are not comparable across models, while intra-model comparisons remain valid.

V. RESULTS

We present our experimental results by analyzing five different aspects of skeptical tools which pertain to design decisions, status recovery, generalizability, verification reliability and grading validity.

A. Analysis of Design Decisions

Tool Selection Accuracy: We begin with assessments on whether our design decisions influence the model’s ability to select the correct tool. The results indicate no significant effect; Qwen 3 32B selected the expected tool in 86–89% of queries across all configurations. This consistency suggests that tool selection is primarily dependent on the model’s inherent capabilities rather than server-side interventions. Further analysis of Config C reveals that many non-matching selections were not errors. Of the 370 tool choices evaluated (74 queries $\times$ 5 runs), 43 deviated from the ground-truth expectation. However, 26 of these were valid refusals or clarification requests where the model appropriately declined invalid requests (e.g., “Prove my data in quantum computing”) or sought clarification for ambiguous inputs (e.g., “Do something with KS”). Only 17 instances constituted genuine errors, such as invoking a tool for an unsupported operation. Table IV presents the detailed results for Qwen 3 32B.

Parameter Correctness: When Qwen 3 32B supplied a parameter, our validation logic detected no errors (100% accuracy across all configurations). However, in status checks, the model frequently omitted the job ID and proof type. This omission is expected, as users often issue vague queries such as “check my status,” and Config C operates without

conversation history to retrieve these values. Consequently, `check_status` calls lacked a proof type in 62 of 86 cases for Qwen, 59 of 108 for Llama, and 44 of 83 for Mistral; similarly, job IDs were omitted in 76 of 86, 84 of 108, and 70 of 83 cases, respectively. The server mitigates these omissions by resolving both values via the job-ID fallback (B4) and stored proof type (B5). This server-side lookup is critical for successful status retrieval and explains why disabling B5 leads to the routing errors described in Section V-B.

Task completion: Task completion is the share of queries that the system completes correctly. For Qwen it rises with each configuration, from 70.2% in Config A to 84.5% in B, 90.3% in C, and 93.0% in D. Config B turns on three design decisions at once (A3, B1, and B4) and Config C turns on two more (B5 and B6), so these numbers show how much each group of decisions helps, not each decision on its own. We find that only Config D adds a single change, the reply templates.

Impact of Identity Injection: The largest performance jump occurs between Config A and Config B, driven by authenticated queries. On login-independent queries Qwen 3 32B holds steady at 92.9–93.5% across all configurations; on authenticated queries it rises from 47.5% in Config A to 77.5%, 88.4%, and 93.0% in Configs B, C, and D respectively (Source 7 excluded). In Config A, disabled identity injection (A3) causes the server to respond “Please sign in first” despite active sessions, leaving models to fabricate placeholders such as “current_user_id” and “user123”. Because identity determines dataset binding, such hallucinations risk linking proofs to the wrong user, motivating A3 from Config B onward.

Robustness to Reference Job Absence: To ensure that the observed performance gap between Config A and subsequent configurations is not artificially inflated by the absence of a reference job in Config A, we conducted an ablation analysis. Specifically, we excluded 21 queries requiring the reference job and re-evaluated the configurations. We note that if the missing reference job were the primary driver of Config A’s lower performance, excluding these queries should have reduced the performance delta between Config A and Config C. On the other hand, the improvement from Config A to Config C increased for all three models (e.g., Qwen 3 32B improved by 23.1 percentage points, up from 20.1). This confirms that the reference job’s absence does not exaggerate the reported improvements, validating the robustness of our findings.

Prompt-Based Identity Injection (Config A):* Config A* supplies the user identity via system prompt rather than backend injection, isolating the effect of A3. All three models propagated the correct identity reliably: Qwen 3 32B and Mistral were correct on every authenticated call (123 and 108 instances respectively), and Llama on 113 of 118, with the five exceptions self-corrected within the same turn. No model hallucinated an identity. Task completion on login-required queries rose from 47.5% in Config A to 85.3%, approaching Config C’s 88.4%; the residual gap reflects the

TABLE V

CONFIG B STATUS QUERIES WHOSE FIRST TOOL CALL RETURNED 404 BECAUSE THE PROOF TYPE WAS OMITTED AND THE TOOL DEFAULTED TO THE WRONG PROVER PORT, FOR TURNS WHERE THE USER’S JOB EXISTED (EXCLUDING THE FIVE NONEXISTENT-JOB CONTROLS AND THE FIVE QUERIES THAT NAMED THE WRONG TYPE). THE LAST THREE ROWS CLASSIFY EACH STOPPED REPLY BY ITS OPENING SENTENCE INTO ONE EXCLUSIVE CLASS.

	Qwen3 32B	Llama 8B	Mistral 24B	Qwen2.5 72B	Total
Wrong-port turns	68	70	60	69	267
Stopped after 1 call	53	70	60	69	252
Retried	15	0	0	0	15
Denied the job exists	48	0	54	0	102
ID invalid or not found	5	50	5	20	80
Vague (“an issue”)	0	20	1	49	70

absence of B5 and B6. Despite this, server-side injection (A3) remains preferable wherein prompt-supplied identities receive no server-side validation and are transmitted to third-party model providers with every request, exposing user identifiers outside the proof system’s privacy boundary.

Implications of Conversation History Retention: Retaining conversation history in Config A* carries one benefit and two costs. On multi-step workflows (Source 6) Qwen 3 32B reached 90.8%, above the 87.5% of Config C, because it could recover job identifiers and proof types from earlier turns. Against this, in 3 of 29 status checks within those workflows it failed to recover the proof type and queried the wrong prover. More consequentially, Config A* was the only configuration in which any model fabricated a result (Section V-C), indicating that accumulated history supplies the context from which hallucination draws.

B. Impact of Stored Proof Types on Status Retrieval Accuracy

In this section, we investigate the ability of LLM agents to correctly retrieve and utilize stored proof types for accurate status queries. We identify a critical failure mode in the CoSMETIC architecture where the absence of server-side context leads to systematic routing errors, highlighting a central mechanistic result of our work. The system employs distinct provers for each statistical test, with each prover hosted on a separate network port. Consequently, the `check_status` tool requires the correct proof type to route requests to the appropriate endpoint. In Config B, where conversation history is disabled and the stored proof type feature (B5) is inactive, ambiguous queries such as “check my status” arrive without an explicit proof type. Lacking this context, the tool defaults to a hardcoded value (`logistic_accuracy`), resulting in misrouted requests to ports that do not host the user’s job. Table V quantifies the frequency of these routing failures across multiple models.

Among the 267 queries that encountered wrong-port routing while the user’s job existed, the model terminated after a single failed call in 252 instances (183 of 198 for the three models tested across all configurations). Only Qwen 3 32B attempted further recovery, retrying in 15 cases. Notably, none of the 252 terminated responses falsely claimed success, consistent with

findings that models accurately report signaled failures [9]. However, these responses were factually incorrect regarding the user’s account status, as the job existed within the system.

We classified the 252 replies by their opening statements and noted that 102 denied the job’s existence (a false statement, as the tool only reported the ID was unknown to the queried port), 80 stated the ID was invalid or not found, and 70 were vague. This distribution varied by model with Qwen 3 32B and Mistral frequently denying existence (48 and 54 times, respectively), whereas Llama and Qwen 2.5 72B never issued such denials. These errors illustrate that while the HTTP 404 signal accurately reflected the queried port’s state, it failed to convey the actual cause of failure. Consistent with prior observations on MCP failure results [26], the system provided no indication of the root cause or potential repair strategies, such as re-querying with the stored proof type. This behavior mirrors the unintended default behaviors identified by Owotogbe et al. in MCP servers [23], where an omitted parameter triggered a silent fallback rather than an explicit error handling mechanism.

Activating the stored proof type feature (B5) in Config C eliminates routing errors caused by omitted parameters. For Qwen 3 32B, the number of status queries resulting in 404 errors decreased from 78 out of 88 to 10 out of 86. The remaining ten failures are attributable to two specific control conditions wherein five involve nonexistent jobs in which a “not found” response is correct; and five involve explicit mismatches between the requested proof type (LRT) and the actual job type (KS), where the model’s input is correctly passed through to the prover. Crucially, B5 resolves all wrong-port routing instances stemming from missing proof types. Across the three primary models, such errors dropped from 198 (Qwen: 68; Llama: 70; Mistral: 60) to zero. Consequently, all remaining 404s in Config C correspond exclusively to these two control sets.

The remaining five errors highlight a divergence in tool design between status retrieval and proof access. The `check_status` tool preserves user input, honoring the model’s explicit (albeit incorrect) proof type selection. In contrast, `download_proof` and `verify_proof` employ an override mechanism that discards the model’s value in favor of server-stored records. This override ensures reliability only when the key used to retrieve the stored record is itself accurate.

We observed this limitation with Llama, which hallucinated job IDs (12345) in 25 download calls across Configs B, C, and D. Although the system’s interceptor replaced the hallucinated session ID (abc123) with the correct one, it could not replace the fabricated job ID. Consequently, the override mechanism failed to find a matching stored record, and the fallback logic (B4) did not trigger because a job ID was technically present. All 75 such attempts failed, underscoring that server-side overrides cannot compensate for hallucinated lookup keys.

Finally, we note the efficiency gains from these improvements. Config C reduced the average number of tool calls per

TABLE VI

WITHIN-MODEL ABLATIONS (%). CORRECTED-GRADER FAITHFULNESS IS 100.0 IN EVERY CELL EXCEPT QWEN A* (99.7). MISTRAL’S B, C, D, AND A* SWEEPS RAN WITH A REFERENCE JOB THAT NEVER REACHED DONE; ITS STATUS AND DOWNLOAD-DEPENDENT CELLS ARE NOT COMPARABLE TO QWEN’S OR LLAMA’S.

Metric	Model	A	B	C	D	A*
Tool sel.	Qwen 32B	86.3	89.0	88.4	88.6	—
	Mistral 24B	79.4	82.0	80.3	81.0	—
	Llama 8B	74.1	77.4	77.8	77.3	—
Task compl.	Qwen 32B	70.2	84.5	90.3	93.0	88.5
	Mistral 24B	65.1	79.7	80.5	88.4	81.2
	Llama 8B	54.9	69.3	68.1	78.1	66.1
Tool use	Qwen 32B	100.0	100.0	100.0	100.0	—
	Mistral 24B	74.0	72.0	76.0	78.0	—
	Llama 8B	80.0	80.0	80.0	80.0	—

status turn from 1.44 (Config B) to 1.00, and decreased average latency from 16.6 seconds to 12.4 seconds, demonstrating that robust context injection minimizes both computational overhead and response time.

The trust label (C1) presents a critical ambiguity. All 252 single-call terminations included the tag [Backed by COSMeTIC prover]”, which verifies tool *invocation* rather than computational *success*. Consequently, this label inadvertently legitimizes failed queries, such as those returning 404 errors. This issue could be resolved by implementing an outcome-based tiering system, where errored-out tools are excluded from receiving the backed” designation. Such a mechanism would serve as a server-side analogue to the named failure states identified by Sethi et al. [9] as the most effective method for communicating execution status.

C. Cross-Model Generalization

Table VI presents the within-model ablations for Qwen, Mistral and Llama family of models. We uncover five key findings from these experiments pertaining to task-completion gains, recovery and retries under exceptions, fallbacks under tool error, failure modes and fabrication chances.

In terms of task completion gains, the performance improvement from Config A to Config D is consistent across all evaluated models, with task completion gains of +22.8% for Qwen 3 32B, +23.2% for Llama, and +23.3% for Mistral. These within-model differences remain robust against the fixture confound, which shifts any Mistral configuration by at most three percentage points.

Following a wrong-port 404 error, recovery attempts varied significantly by model family. Both Qwen 3 models attempted further tool calls after initial failures. While Qwen 3 32B retried in 15 of 68 turns, Qwen 3 235B retried in 6 of 19 first-call 404s during its single complete run. In contrast, Llama, Mistral, and Qwen 2.5 72B never attempted retries (0 of 199 turns). While it remains untested whether “thinking” modes or inherent model generation capabilities drive this tendency, these results contrast with Tian et al.’s findings on their benchmark [27] as well. When Qwen 3 32B did retry, it typically re-queried the live API. However, in four instances,

it presented cached Redis status as current despite a failed live check, ignoring the tool’s staleness warning. This constitutes the B3 failure mode, representing a stale-value scenario where Sethi et al. [9] also observed dishonesty, as cached values lack inherent failure signals.

In stateless configurations, server-side fallbacks proved ineffective for models that failed to invoke tools. For Source 6 (Table II) download steps, Mistral and Qwen 2.5 72B requested job IDs from users without calling the tool (Mistral did so in 10 of 10 cases across Configs B, C, and D). Conversely, Qwen 3 32B invoked the tool in all 10 cases. Thus, server-side fallbacks assist only those models willing to call the tool without explicit parameters; some models simply do not make the call.

On Source 4’s (Table II) impossible requests, Qwen and Llama frequently substituted the nearest available tool (9–11 of 25 requests per configuration), such as mapping “export to PDF” to `download_proof`. Mistral, however, never substituted tools; instead, it promised non-existent capabilities (occurring 0, 4, 6, 0, and 8 times in Configs A, B, C, D, and A*, respectively). The zero occurrences in Config D resulted from template-based response overrides. On Source 7, Qwen called a tool on every query, whereas Mistral and Llama did so in 72–78% and 80% of cases, respectively.

The only fabricated outcomes in the dataset were seven zero-tool replies, all occurring in Config A* and exclusively from non-thinking models. Llama called no tool in 5 of its 10 A* download steps, replying that proofs had been successfully downloaded. Similarly, Mistral claimed job resubmission in 2 of its 5 A* repeat steps, once providing a fabricated job ID absent from any tool output. Although Config A also retained history, it produced no fabrications because no prior successes existed to imitate. This suggests fabrication requires both history and a precedent of success, providing evidence for the role of B1. This aligns with Wu’s production analogue, where persistent alerts in chat history led to invented follow-up tasks [28], and Toda and Mori’s security analysis agent, which avoids passing accumulated history to mitigate context poisoning [10].

D. End-to-End Proof Lifecycle and Verification Reliability

Table VII illustrates a complete proof lifecycle executed via natural language interactions in Config D using Qwen 3 32B. The `verify_proof` tool invokes CoSMETIC’s verification endpoint, leveraging the same B4/B5/B6 context stack as `download_proof`. In Config D, this endpoint returns a binary outcome: Proof verification PASSED” or FAILED”.

We evaluate the verification extension on Qwen 3 32B in all four configurations and on Mistral 24B in Configs C and D, giving 35 records per model and 210 in total. The Llama runs were incomplete and are excluded. Qwen completed the full lifecycle in all five runs of Configs C and D, invoking all four tools with verification passing each time (Table VII), and its task completion rose from 60% in Config A to 80%, 94%, and 100% in B, C, and D. Mistral never completed it, making no download call and asking the user for the job ID as in the main suite, and selecting `verify_proof` in only 7 of 20 turns in

TABLE VII
FULL LIFECYCLE THROUGH NATURAL LANGUAGE (CONFIG D, QWEN 3 32B), TRANSCRIBED FROM THE LOGGED RECORD FOR JOB 1778716348432. LABELS ARE APPENDED BY SERVER-SIDE CODE AND REPRODUCED AS LOGGED.

Step	Interaction
User	“Prove my data in KS”
System	“Your proof has been submitted. Job ID: 1778716348432. You can check the status by asking ‘check my status.’” [Backed by COSMeTIC prover]
User	“Check my status”
System	“Job 1778716348432: Status is done.” [Backed by COSMeTIC prover]
User	“Download my proof”
System	“Your proof file has been downloaded. Filename: proofs-1778716348432.zip, size: 378964 bytes. Use the download button to save it.” [Backed by COSMeTIC prover]
User	“Verify my proof”
System	“Proof verification PASSED.” [Backed by COSMeTIC prover]

Config C and 9 of 20 in D. In Config B, `verify_proof` produced zero wrong-port 404s and no false PASSED results across 24 calls. The reason is not B5, which is inactive there, but the absence of a default, since the tool returns “proof_type is required” rather than guessing a port. `check_status` instead defaults to `logistic_accuracy`, routing silently to port 5012 and producing the 404s of Section V-B.

It is important to note that cryptographic verification cannot detect semantic mismatches upstream. For instance, if a user intended an LRT proof but generated a KS proof due to earlier confusion, `verify_proof` would still return “PASSED” because it validates the proof against the prover that generated it. Therefore, upstream protections, specifically proof-type normalization (A1) and stored proof type retrieval (B5), are critical for ensuring end-to-end fidelity, outweighing the role of downstream verification alone.

Regarding output transparency, the evaluated build emitted only two generic trust labels. Across 4,862 logged turns, responses carried [Backed by COSMeTIC prover]”, while 1,476 carried [Not backed by COSMeTIC prover]”. Thirty-three turns carried no label. Consequently, Table VII displays the generic “backed” label following a passed verification, as the more granular cryptographic tiers defined in the framework were not exercised in these experimental runs.

E. Grading Validity and Metric Robustness

The original grader reported 83.7% to 100% across the primary models, giving Qwen 3 32B 93.2%, 83.8%, 97.7%, 99.0%, and 88.4% in Configs A through D and A*. The corrected grader reports 100% everywhere except Qwen in Config A* (99.7%), and the difference is entirely grader artifact. Hand-reading all 47 responses flagged for Qwen in Config B found no ungrounded success claim, as 32 were correct error reports penalized for containing “submitted” and 8 were successful recoveries after a 404. An extended review

of 151 flagged records showed the same pattern, the original grader scoring a third-call recovery as a fabrication from the first. Read at face value its scores would rank Qwen least reliable in Config B (83.8%) against Llama (93.8%) and Mistral (94.7%), yet the transcripts show no fabrication. A systematic search of all 24 result files found only the seven zero-tool fabrications of Section V-C.

These seven fabrications expose a gap in standard faithfulness metrics. When no tool is invoked, faithfulness is undefined, yet five of the seven passed task completion because replies contained keywords like download” and proof.” Only tool selection accuracy and the trust label correctly flagged them as [Not backed by CoSMETIC prover]. A substring grader also over-reports success in the opposite direction, crediting Mistral with completing the lifecycle in 5 of 5 runs despite Mistral explicitly stating it lacked the tools to verify. We therefore define lifecycle completion strictly by tool invocation, not keyword presence. A related fixture blind spot is that the substring grader passed “you cannot download the proof until the job is done” as a task completion, but outcome counts confirm zero successful downloads in those runs.

VI. DISCUSSION

A. Correctness Lives at the Tool Boundary

At a fundamental level, we find that any agent architecture that routes tool calls through parameterized endpoints faces analogous structural risks. When a model omits or incorrectly defaults a routing parameter, the tool queries an incorrect backend, rendering any subsequent response factually inaccurate for the user. The 252 single-call terminations analyzed in Section V-B were not fabrications; rather, they resulted from upstream routing errors. Consequently, model honesty cannot rectify misrouted requests.

These findings necessitate a redefinition of “skeptical” design principles in agent systems. We enumerate three distinguishing mechanisms for handling parameter uncertainty. *Validation* rejects invalid parameters, returning an error that models typically relay to the user. *Completion* infers missing parameters from server-side state, resolving the error before it reaches the model but potentially retaining explicitly incorrect user inputs. *Override* discards the model’s input in favor of authoritative server-side state, eliminating both the error and the risk of misinformation. Our implementation incorporates all three strategies. The residual routing errors observed in Config C highlight the limitations of *completion* relative to *override*. Specifically, when fallback logic is disabled, `check_status` defaults to a single prover, generating confident 404 errors from incorrect ports. In contrast, `verify_proof` lacks a default value, returning recoverable errors and avoiding misrouting entirely. This suggests that providing a plausible default is more hazardous than providing no default, as it transforms missing information into confidently incorrect destinations. This phenomenon mirrors observations by Mousavi et al. regarding LLM-generated security code, where default cipher modes contributed to misuse rates [29]. Finally, we propose that MCP tool developers adopt an *override* whenever server-side state is authoritative. In other

TABLE VIII
THREAT MODEL: ZKSNARK SPECIFIC ASSETS, THE LLM BEHAVIOR THAT
ENDANGERS EACH, THE MITIGATION, AND WHERE IT IS EVALUATED.

Asset	LLM threat	Mitigation	Evaluated
Circuit binding	Omitted <code>proof_type</code> routes to the wrong analysis	A1, B5	Wrong-port routing (§V-B)
Dataset binding	Placeholder <code>user_id</code> binds the proof to the wrong SMT commitment	A3	Config A vs. A*
Proof lineage	Job ID lost across stateless turns	B1, B4	Source 6
Proof status	Cached status presented as current	B3, B5	Warning ignored in 4 turns (§V-C)
Proof validity	Validity inferred from status without verification	<code>verify_proof</code>	Lifecycle (§V-D)
Provenance	Zero-tool success claim indistinguishable from a grounded one	C1 labels	Grading validity (§V-E)
Metadata	Hashes, job IDs, and account UUID in tool output reach the model provider	None; display-side scrub only	Not mitigated

words, if the server possesses the correct value for a routing parameter, it should disregard the model’s input altogether [9].

B. Implications for Zero-Knowledge Deployments

Three properties separate zero-knowledge deployments from conventional tool calling, each posing a threat summarized in Table VIII. First, proofs are permanent. A proof shared with a verifier is an irreversible claim, so a user wrongly informed about the absence of a valid job may resubmit and create duplicate proofs over one dataset. Second, circuit selection is semantically binding. A misrouted status query returns a detectable 404, but a misrouted submission fails silently, yielding a valid proof of an unintended computation. A user requesting LRT could receive a KS proof that verifies yet establishes the wrong property. Once such an error enters the cryptographic layer it is undetectable, so it must be prevented upstream. We observed none, as all prove calls carried canonical types, but the risk is architectural. Third, identity determines the dataset commitment, making identity injection a cryptographic binding requirement rather than an authentication convenience.

The final row of Table VIII highlights a critical privacy consideration. While the underlying proof system retains participant data within the provers, the interface transmits dataset hashes, job identifiers, proof types, and account UUIDs to the model provider with every request. Since prompt templates do not alter the information already processed by the model, the provider resides within the trust boundary for clinical deployments unless tool outputs are scrubbed before reaching the model. In the evaluated system, scrubbing occurs only at the display boundary, leaving intermediate requests exposed.

C. Architecture, Model Capability, and Summarization

Tool selection accuracy scales with model capability, reaching approximately 88% for Qwen, 80% for Mistral, and 77% for Llama. This metric remains largely independent of the specific decisions under study. Behaviors such as retrying after an error, invoking a tool without a required parameter, or substituting tools for unsupported requests vary by model architecture rather than configuration. Category B fallbacks assist Qwen because it tends to invoke the tool regardless; however, they are ineffective for Mistral, which prefers to query the user for missing parameters.

Configs C and D achieve near-identical tool selection performance, differing by no more than 0.7 points across any model. This similarity isolates the impact of LLM summarization capabilities. Responses under Config C are 78% longer than those under Config D (349 versus 196 characters on tool turns) and exhibit significantly lower repeatability. Specifically, under Config D, 34 of Qwen’s 75 queries yield byte-identical responses across five runs, whereas only one such instance occurs under Config C. Consequently, Config D demonstrates a task-completion advantage over Config C of +2.7 for Qwen, +7.9 for Mistral, and +10.0 for Llama. These results suggest that templating provides the greatest benefit where models exhibit weaker summarization abilities. However, Config D introduces specific costs wherein hash-existence outputs and error payloads pass through unformatted, and only the final tool call is templated. Furthermore, in 67 of Qwen’s 375 Config D turns, the model made no tool call and returned raw text instead.

D. Design Guidelines for MCP Integration

Our study yields five guidelines for efficient, secure tool-boundary design. First, routing parameters should be overridden from authoritative state, so that the server discards model-supplied values wherever it already knows the correct backend. This is safe only when the key selecting the stored record is itself trustworthy, so a supplied job ID should be checked against the active session. Second, identity should be stripped and injected server-side, because a model-supplied identity is accepted on the model’s word alone and models fill the slot with placeholders when given no channel, making injection a security boundary rather than a convenience. Third, conversation history should be withheld from tool-grounded operations, since a history containing a successful prior turn was the only condition under which any model fabricated an outcome. Fourth, trust labels should vouch for outcomes rather than invocations, which requires an outcome tier so that an erroneous tool never earns the label, paired with a metric that flags success claims on turns with no tool call. Lastly, tool outputs should be scrubbed before the model sees them and the boundary must be tested against every deployed model. This is because tool return values rather than templates determine the metadata reaching the provider, and fallback behavior varies sharply across models.

E. Limitations

Several limitations qualify these results. The 15 always-on decisions are integral to the architecture and were assessed

by inspection and code review rather than ablation. We ran no configuration pairing history retention with identity injection, and the Config A fixture asymmetry understates rather than inflates the architecture’s contribution. Because reference jobs never completed in the Mistral, Qwen 2.5 72B, and Qwen 3 235B sweeps, the download benefit of B5 and B6 rests on Qwen and Llama evidence. An invented job ID defeats both B4 and B5, and we tested no unsigaled failures such as stale or corrupted payloads [9]. Finally, the fabrication finding rests on seven records from two models in one configuration, substring grading ignores tool identity and outcome, and all queries use a single test user on open-weight models alone.

VII. CONCLUSION

Zero-knowledge verifiable computation is reliably accessible through natural language interfaces. Using skeptical tool validation and structured reply templates, our system improved task completion by 23 percentage points over a naive chatbot baseline across all three model families, achieving 93% on Qwen 3 32B and completing the full prove–status–download–verify lifecycle in every Qwen trial. The central finding is that correctness must be enforced at the tool layer rather than delegated to the model. When stored proof-type metadata was disabled, a plausible default routed 198 status queries to the wrong prover, whereas re-enabling server-side override eliminated all misroutes and tools that only fill missing values produced five residual errors, confirming that a plausible default is strictly more hazardous than no default. We further find that hallucination occurred only when conversation history contained a prior success to imitate, motivating stateless tool design; that evaluating agent honesty against only the first tool call produces systematic measurement error in both directions; and that hashes, job IDs, and account identifiers reach the model provider in every configuration, requiring redaction before model ingestion in sensitive deployments. We anticipate this pattern generalizes to any setting requiring precise, verifiable LLM outputs, and broader evaluation remains future work.

ACKNOWLEDGMENT

The authors disclose that generative AI tools (Claude, Anthropic and Qwen-3.6-Thinking) were used in the preparation of this manuscript. The use of these tools was limited to assisting with grammar, syntax, and the polishing of author-written text for fluency.

REFERENCES

- [1] M. Shahid *et al.*, “CoSMeTIC: Zero-knowledge computational sparse Merkle trees with inclusion-exclusion proofs for clinical research,” *arXiv preprint arXiv:2601.12136*, 2026.
- [2] A. Frolov, M. Shih, R. Patro, and I. Miers, “Icefish: Practical zk-SNARKs for verifiable genomics,” *Cryptology ePrint Archive*, Paper 2026/463, 2026.
- [3] J. Zhang, Y. Zhou, Y. Ren, M. H. Au, K.-H. Chow, L. Chen, Y. Zhao, J. Su, and R. Luo, “Toward owner governance in genomic data privacy with governance,” *Cell reports methods*, vol. 5, no. 9, 2025.
- [4] F. Scaramuzza, R. C. Ferreira, G. Quattrocchi, D. A. Tamburri, and W.-J. van den Heuvel, “‘show me you comply...without showing me anything’: Zero-knowledge software auditing for AI-enabled systems,” *arXiv preprint arXiv:2510.26576*, 2025.
- [5] C. Carreira, J. ao F. Ferreira, A. Mendes, and N. Christin, “Exploring usable security to improve the impact of formal verification: A research agenda,” in *Proc. Workshop on Applicable Formal Methods (AppFM)*, vol. 349 of *EPTCS*, pp. 77–84, 2021.
- [6] J. Groth, “On the size of pairing-based non-interactive arguments,” in *Advances in Cryptology – EUROCRYPT 2016*, LNCS 9666, pp. 305–326, Springer, 2016.
- [7] C. Yin *et al.*, “The reasoning trap: How enhancing LLM reasoning amplifies tool hallucination,” in *Proc. ACL (Long Papers)*, 2026.
- [8] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith, “How language model hallucinations can snowball,” in *Proc. ICML*, PMLR 235, pp. 59670–59684, 2024.
- [9] A. Sethi *et al.*, “Fabrication after tool failure: Tool-augmented agents assert values their tools did not return,” *arXiv preprint arXiv:2609.14758*, 2026.
- [10] T. Toda and T. Mori, “CHASE: LLM agents for dissecting malicious PyPI packages,” in *Proc. AWARE*, 2025.
- [11] S. Yao *et al.*, “ReAct: Synergizing reasoning and acting in language models,” in *Proc. ICLR*, 2023.
- [12] X. Hou *et al.*, “Model context protocol (MCP): Landscape, security threats, and future research directions,” *ACM Trans. Softw. Eng. Methodol.*, 2025.
- [13] S. Goldwasser, S. Micali, and C. Rackoff, “The knowledge complexity of interactive proof systems,” *SIAM J. Comput.*, vol. 18, no. 1, pp. 186–208, 1989.
- [14] B. Parno, J. Howell, C. Gentry, and M. Raykova, “Pinocchio: Nearly practical verifiable computation,” in *Proc. IEEE Symp. Security and Privacy*, pp. 238–252, 2013.
- [15] S. Chaliasos *et al.*, “SoK: What don’t we know? Understanding security vulnerabilities in SNARKs,” in *Proc. USENIX Security Symp.*, 2024.
- [16] H. Sun, J. Li, and H. Zhang, “zkLLM: Zero knowledge proofs for large language models,” in *Proc. ACM CCS*, pp. 4405–4419, 2024.
- [17] Z. Xue *et al.*, “From evaluation to enhancement: Large language models for zero-knowledge proof code generation,” *arXiv preprint arXiv:2509.11708*, 2025.
- [18] H. Watanabe and M. Uchikoshi, “Generating privacy-preserving personalized advice with zero-knowledge proofs and LLMs,” in *Proc. WWW Companion*, 2025.
- [19] G. Jing and H. Qi, “Zero-knowledge audit for internet of agents: Privacy-preserving communication verification with model context protocol,” *arXiv preprint arXiv:2512.14737*, 2025.
- [20] A. S. Gopalakrishna, “Zero-knowledge predicate proofs between AI agents: A measured, cross-protocol gateway and the source-integrity gap,” *arXiv preprint arXiv:2608.30083*, 2026.
- [21] AffixIO, “AffixIO MCP: Model context protocol connector for zero-knowledge prove and verify,” <https://github.com/AffixIO/MCP>, accessed Sep. 23, 2026.
- [22] M. M. Hasan *et al.*, “Model context protocol (MCP) tool descriptions are smelly! Towards improving AI agent efficiency with augmented MCP tool descriptions,” *arXiv preprint arXiv:2602.14878*, 2026.
- [23] J. Owotogbe *et al.*, “A taxonomy of runtime faults in model context protocol servers,” *arXiv preprint arXiv:2606.05339v2*, 2026.
- [24] G. Sattanathan, R. Byrro, and E. Tahler, “54 patterns for building better MCP tools,” <https://www.arcade.dev/blog/mcp-tool-patterns>. Arcade.dev, Feb. 2026; accessed Sep. 22, 2026.
- [25] V. Reddy, S. R. Challaram, and A. Basu, “Reason less, verify more: Deterministic gates recover a silent policy-violation failure mode in tool-using LLM agents,” in *KDD Workshop on Evaluation and Trustworthiness of Agentic AI*, 2026.
- [26] R. Mehan, “Can MCP clients decide what to do after failure? A result-only actionability audit,” *arXiv preprint arXiv:2609.00072*, 2026.
- [27] Y. Tian, Z. Shi, Y. Zhou, and B. Zhao, “Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability,” *arXiv preprint arXiv:2606.25819*, 2026.
- [28] W. Wu, “When errors become narratives: A longitudinal taxonomy of silent failures in a production LLM agent runtime,” *arXiv preprint arXiv:2606.14589*, 2026.
- [29] Z. Mousavi *et al.*, “An investigation into misuse of Java security APIs by large language models,” in *Proc. ACM ASIA CCS*, pp. 1299–1315, 2024.